

1. Stworzenie opisu projektu
 2. Określenie problemu, cechy wejściowe
 3. Wstępna analiza danych i ich wizualizacja
 4. Sprawdzenie pomocniczych klas, czyli balans danych
 5. Eksperyment 1 - analiza cech
 6. Przygotowanie danych do modelu
 7. Eksperyment 2 - regresja
 8. Walidacja leave-one-out
 9. Ocena modeli za pomocą metryk MAE RMSE, R^2
 10. Analiza błędów
 11. Eksperyment 3 - klasteryzacja
 12. Interpretacja klastrów
 13. Porównanie wyników
 14. Wnioski końcowe
 15. Wizualizacja danych
-

Opis projektu

Predykcja poziomu szczęścia krajów na podstawie wskaźników społeczno-ekonomicznych.

Wykorzystując zbiór danych World Happiness Report 2026, który zawiera informacje o krajach, ich wyniku szczęścia i dodatkowych czynnikach, zostanie stworzony model predykcji poziomu szczęścia krajów.

Problem i cechy wejściowe

Celem jest przewidywanie wartości score, czyli liczbowego wyniku szczęścia kraju.

Typ problemu: uczenie nadzorowane i regresja wielowymiarowa. Zmienna docelowa:

`score` Cechy wejściowe:

```
gdp_per_capita
social_support
healthy_life_expectancy
freedom
generosity
corruption
```

Dodatkowo będzie analizowana zmienna `region`.

Wstępna analiza danych i wizualizacja

Stworzono wizualizacje danych:

- histogram wyniku szczęścia,

- top 10 najszcześniejszych krajów,
- bottom 10 najszcześniejszych krajów,
- zależność szczęścia od PKB,
- zależność szczęścia od wsparcia społecznego,
- średni score według regionu,
- macierz korelacji.

Sprawdzono również rozkład danych. Oprócz `generosity` zmienne są mocno powiązane ze `score`. Nie są zauważalne wartości odstające czy skrajne.

Analizę załączono na końcu tego dokumentu.

Sprawdzenie pomocniczych klas

Zostanie sprawdzony balans danych. Utworzona zostanie zmienna pomocnicza `happiness_level`. Na podstawie wartości score kraje zostaną podzielone na grupy.

Ze względu na to, że problemem jest regresja, niezbalansowanie tych klas nie jest kluczowym problemem dla modelu. Jednak jest to przydatna informacja.

Eksperyment 1 - analiza danych

Celem jest sprawdzenie, które cechy są najbardziej powiązane z wynikiem szczęścia.

W eksperymencie wykonane zostanie:

- korelacja cech ze score,
- korelacja cech między sobą,
- ranking cech,
- analiza istotności cech.

Zostaną wykorzystane metody: `SelectKBest`, `f_regression` oraz `mutual_info_regression`.

Przygotowanie danych do modelu

Zostaną przygotowane dane do uczenia maszynowego, czyli `X` jako cechy wejściowe oraz `y` jako zmienna docelowa score. Tutaj zostanie zdecydowane, czyli model będzie trenowany na wszystkich cechach czy tylko na tych najlepszych z eksperymentu nr 1.

Przygotowanie danych liczbowo:

- usunięcie nieużywanych kolumn,
- sprawdzenie braków danych,
- skalowanie cech.

Eksperyment 2 - regresja

Porównanie kilka modeli regresyjnych, np. linear regression, ridge regression, lasso regression, random forest regressor, decision tree regressor.

Walidacja Leave-one-out

Zostanie wykorzystana walidacja leave-one-out cross-validation, czyli model uczy się na 129 krajach i testuje na 1 kraju, a cały proces powtarza się 130 razy. Jest to dobra metoda na mały zbiór danych.

Ocena modeli za pomocą MAE, RMSE i R^2

MAE – średni błąd bezwzględny

RMSE – średni błąd kwadratowy, mocniej karze duże błędy

R^2 – pokazuje, jaką część zmienności score wyjaśnia model

Porównane zostaną modele.

Analiza błędów

Po wybraniu najlepszego modelu zostaną sprawdzone jego błędy. Tutaj odniesiemy się do pomocniczych klas, żeby sprawdzić, czy błąd modelu różni się między grupami.

Eksperyment 3 - klasteryzacja

Celem jest sprawdzenie, czy kraje można podzielić na grupy o podobnych cechach społeczno-ekonomicznych. Pomoże to znaleźć podobieństwa między krajami.

Wykorzystana metoda: K-Means.

Interpretacja klastrów

Po klasteryzacji, sprawdzi się czym się różnią pojedyncze grupy.

Porównanie wyników

Podsumowanie:

- które cechy miały największy związek ze score,
- który model regresyjny uzyskał najlepsze wyniki,
- czy leave-one-out był dobrym wyborem,
- czy klasteryzacja pokazała sensowne grupy krajów,
- czy wyniki regresji i klasteryzacji są ze sobą spójne.

Wyniki końcowe

Czyli wnioski

Wizualizacja analizy danych

Analizowano zbiór danych World Happiness Report 2026, który zawiera informacje o krajach, ich wyniku szczęścia i dodatkowych czynnikach

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('world_happiness_2026.csv')

data = df.to_numpy()
score = data[1:, 3]
score = score.astype(dtype=float)
```

```
In [2]: pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)
pd.set_option("display.max_colwidth", None)
pd.set_option("display.expand_frame_repr", False)
```

```
In [3]: plt.hist(score)
plt.title("Histogram wyniku szczęścia")
plt.xlabel("Score")
plt.show()
```



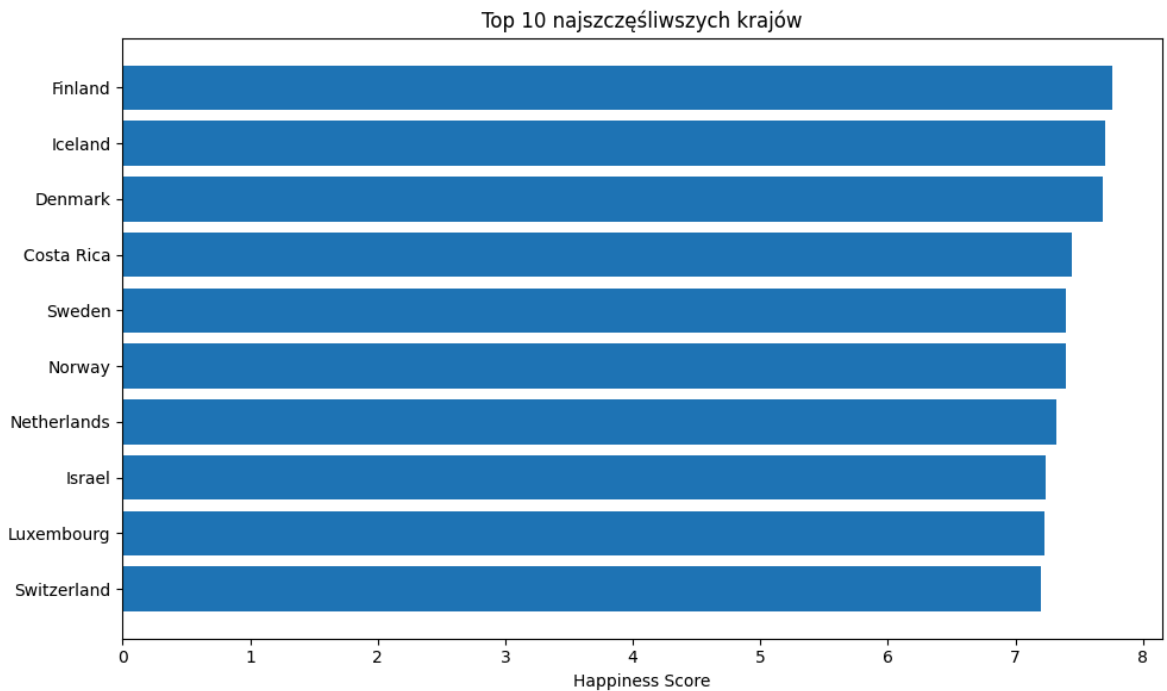
```
In [4]: top10 = df.sort_values(by="score", ascending=False).head(10)
bottom10 = df.sort_values(by="score", ascending=True).head(10)

plt.figure(figsize=(10, 6))
plt.barh(top10["country"], top10["score"])

plt.title("Top 10 najszczęśliwszych krajów")
plt.xlabel("Happiness Score")
```

```
plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```

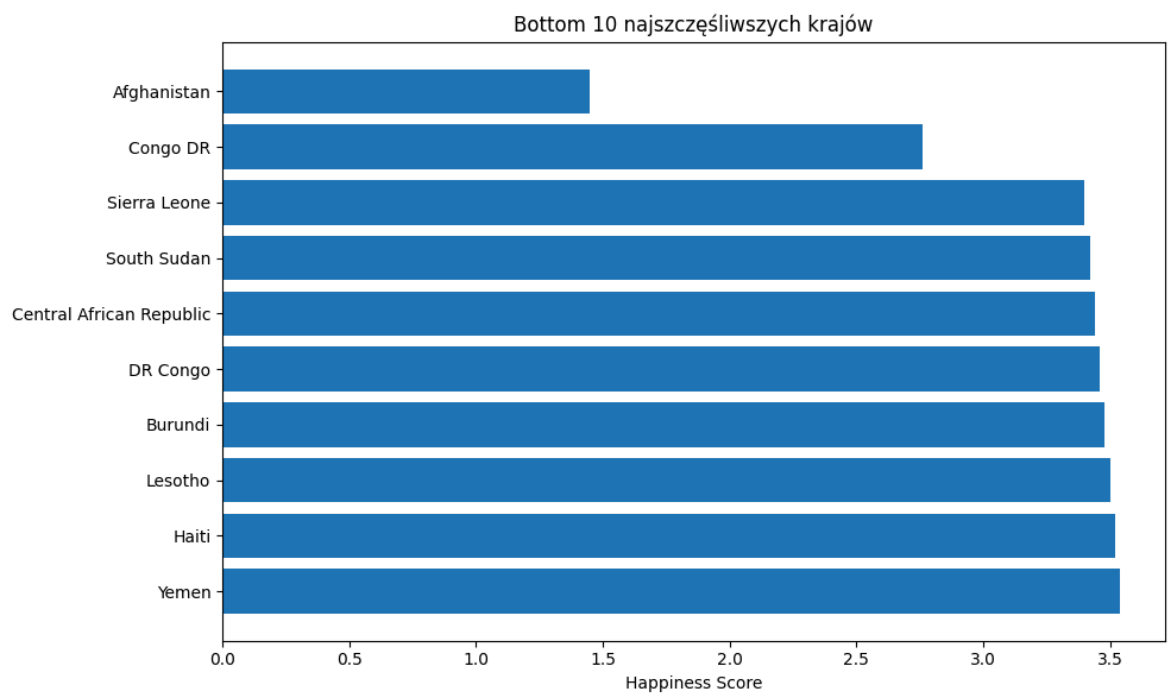


```
In [5]: plt.figure(figsize=(10, 6))
plt.barh(bottom10["country"], bottom10["score"])

plt.title("Bottom 10 najszczęśliwszych krajów")
plt.xlabel("Happiness Score")

plt.gca().invert_yaxis()

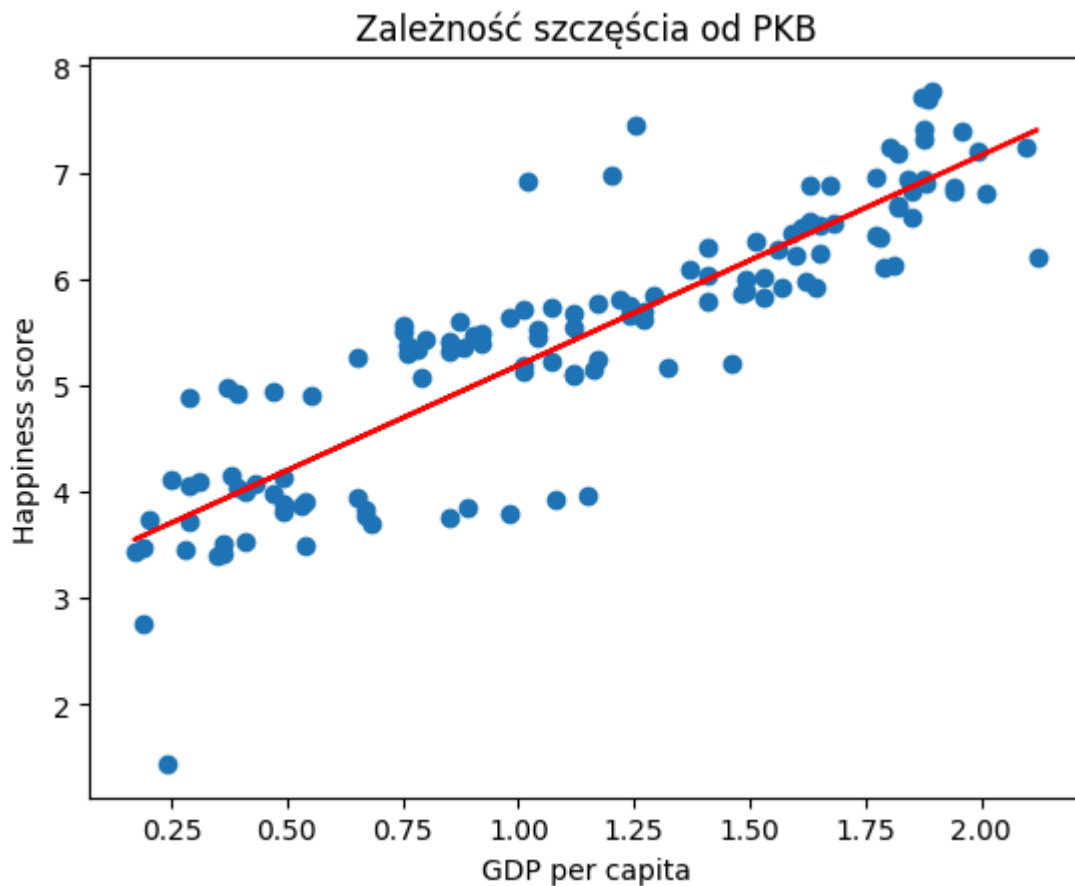
plt.tight_layout()
plt.show()
```



```
In [6]: plt.scatter(df["gdp_per_capita"], df["score"])

m, b = np.polyfit(df["gdp_per_capita"], df["score"], 1)
plt.plot(df["gdp_per_capita"], m*df["gdp_per_capita"] + b, color="red")

plt.xlabel("GDP per capita")
plt.ylabel("Happiness score")
plt.title("Zależność szczęścia od PKB")
plt.show()
```



```
In [7]: plt.scatter(df["social_support"], df["score"])

m, b = np.polyfit(df["social_support"], df["score"], 1)
plt.plot(df["social_support"], m*df["social_support"] + b, color="red")

plt.xlabel("GDP per capita")
plt.ylabel("Happiness score")
plt.title("Zależność szczęścia od wsparcia społecznego")
plt.show()
```



Wykres zależności szczęścia od wsparcia społecznego pokazuje zależność liniową.

```
In [8]: mean = df.groupby(["region"])[ "score" ].mean().sort_values()

print(mean)

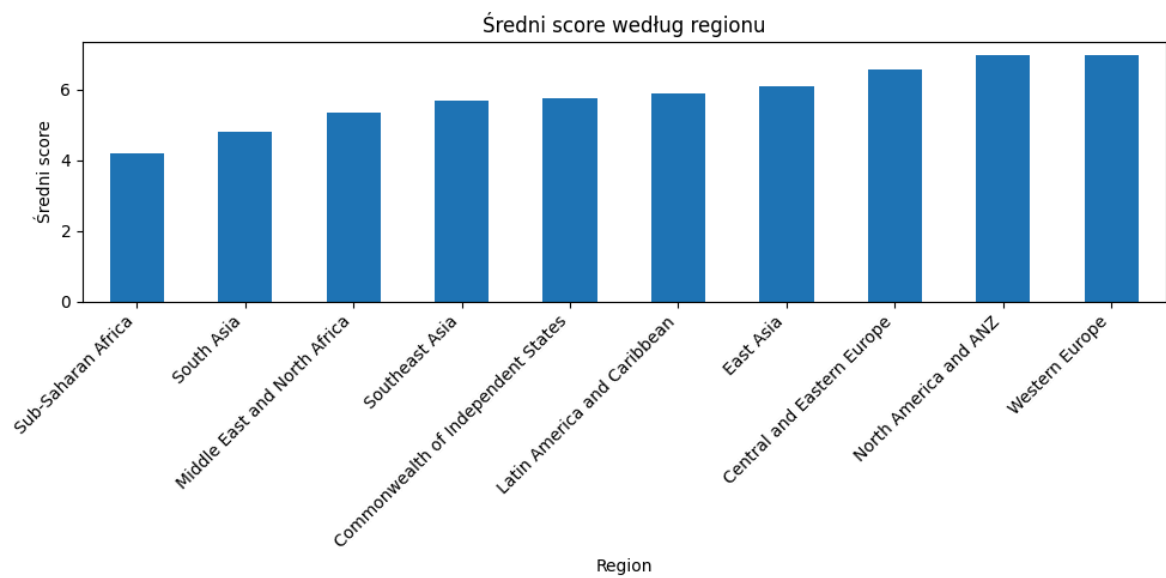
mean.plot(kind="bar", figsize=(10,5))

plt.title("Średni score według regionu")
plt.xlabel("Region")
plt.ylabel("Średni score")
plt.xticks(rotation=45, ha="right")

plt.tight_layout()
plt.show()
```

region	
Sub-Saharan Africa	4.204256
South Asia	4.816833
Middle East and North Africa	5.356929
Southeast Asia	5.694333
Commonwealth of Independent States	5.757000
Latin America and Caribbean	5.874933
East Asia	6.099200
Central and Eastern Europe	6.551273
North America and ANZ	6.969000
Western Europe	6.987167

Name: score, dtype: float64



```
In [9]: corr = df.corr(numeric_only=True)

fig, ax = plt.subplots(figsize=(8,6))

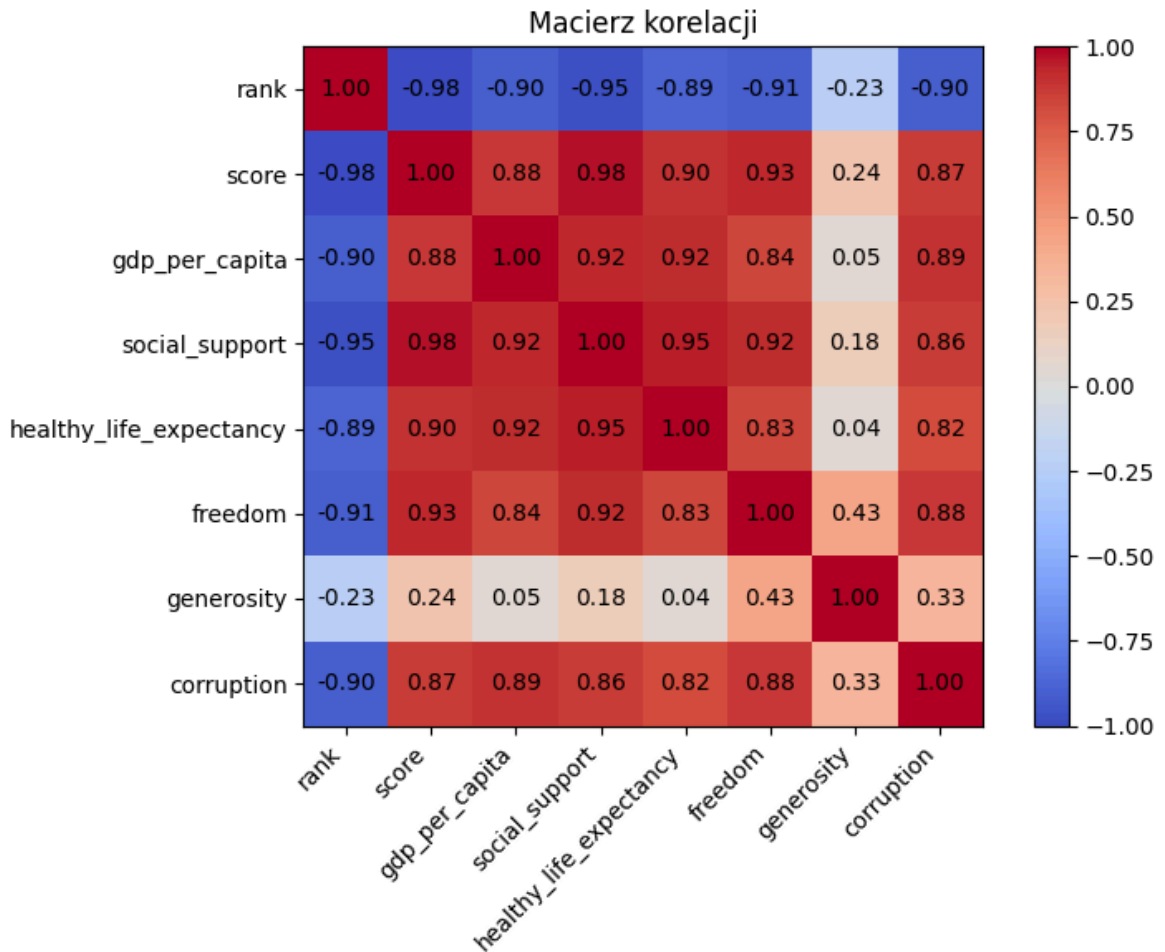
cax = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)

ax.set_xticks(np.arange(len(corr.columns)))
ax.set_yticks(np.arange(len(corr.columns)))

ax.set_xticklabels(corr.columns, rotation=45, ha="right")
ax.set_yticklabels(corr.columns)

for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}",
                ha="center", va="center", color="black")

plt.colorbar(cax)
plt.title("Macierz korelacji")
plt.tight_layout()
plt.show()
```

Macierz korelacji pokazuje mocną zależność parametrów od wskaźnika "score".

```
In [10]: df["happiness_level"] = pd.cut(df["score"], bins=3, labels=["Low", "Medium", "High"])
print(df["happiness_level"].value_counts())
```

```
happiness_level
High      61
Medium    59
Low       10
Name: count, dtype: int64
```

Dane są niezbalansowane, ale przedstawiają stan rzeczywisty. Będziemy wykorzystywać modele regresyjne, dlatego nie możemy na to wpłynąć.

Eksperyment 1 - analiza cech

Celem eksperymentu jest sprawdzenie, które cechy wejściowe są najmocniej powiązane ze zmienną docelową `score`. Analiza obejmuje korelacje liniowe, zależności między cechami oraz ranking cech wyznaczony metodami `SelectKBest`, `f_regression` i `mutual_info_regression`.

```
In [11]: from sklearn.feature_selection import SelectKBest, f_regression, mutual_info_regression

features = [
    "gdp_per_capita",
    "social_support",
    "healthy_life_expectancy",
```

```

    "freedom",
    "generosity",
    "corruption",
]

X = df[features]
y = df["score"]

```

```

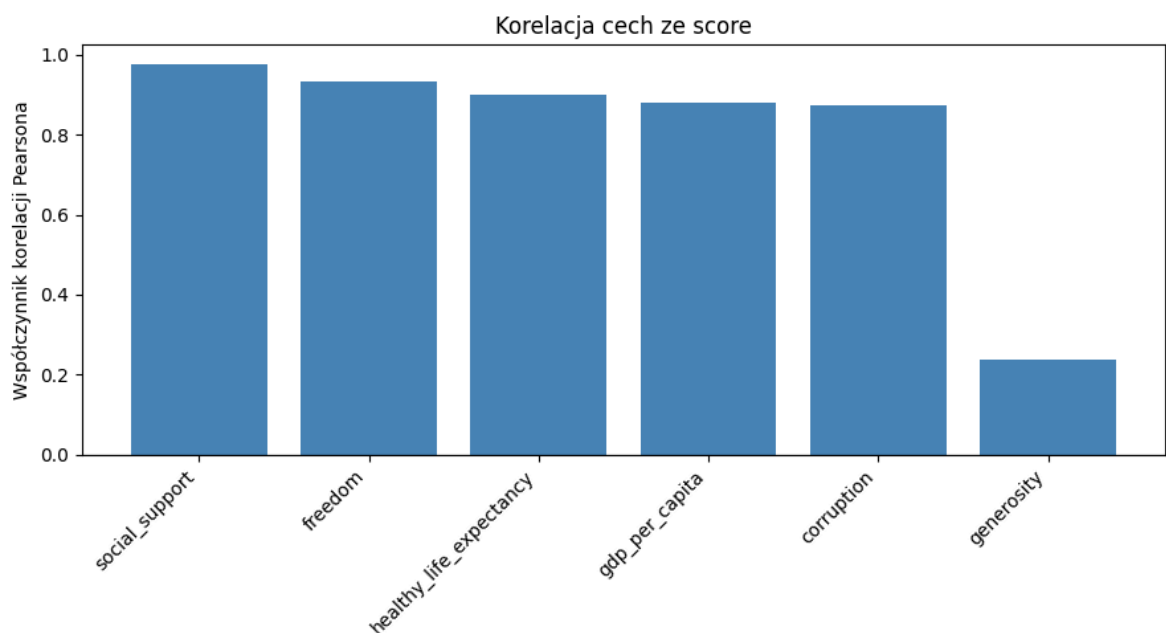
In [12]: corr_with_score = X.corrwith(y)
corr_ranking = pd.DataFrame({"correlation_with_score": corr_with_score}).sort_va

display(corr_ranking)

plt.figure(figsize=(9, 5))
plt.bar(corr_ranking.index, corr_ranking["correlation_with_score"], color="steel
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Korelacja cech ze score")
plt.ylabel("Współczynnik korelacji Pearsona")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```

correlation_with_score	
social_support	0.976465
freedom	0.932961
healthy_life_expectancy	0.900567
gdp_per_capita	0.879284
corruption	0.874750
generosity	0.236826



```

In [13]: feature_corr = X.corr()

fig, ax = plt.subplots(figsize=(8, 6))
cax = ax.imshow(feature_corr, cmap="coolwarm", vmin=-1, vmax=1)

```

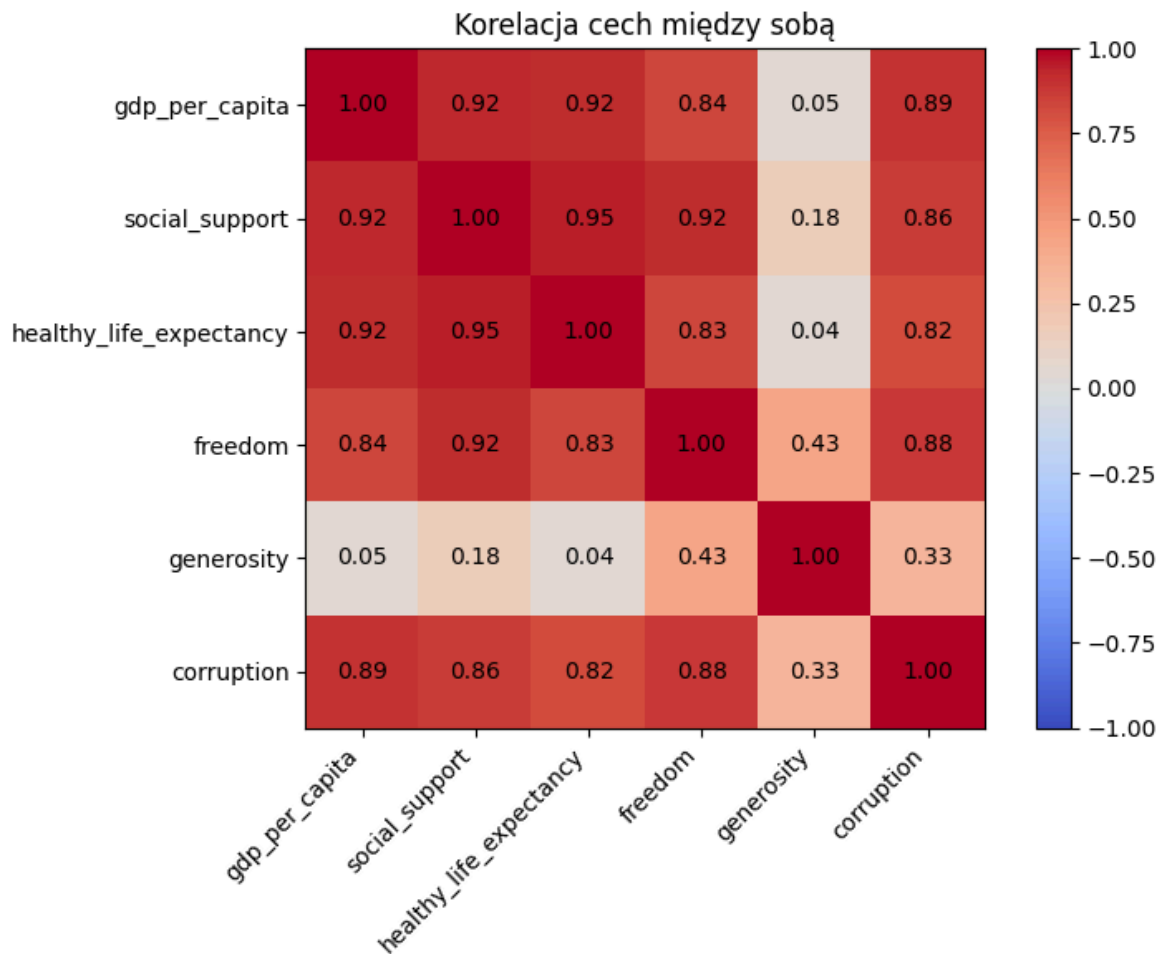
```

ax.set_xticks(np.arange(len(features)))
ax.set_yticks(np.arange(len(features)))
ax.set_xticklabels(features, rotation=45, ha="right")
ax.set_yticklabels(features)

for i in range(len(features)):
    for j in range(len(features)):
        ax.text(j, i, f"{feature_corr.iloc[i, j]:.2f}", ha="center", va="center")

fig.colorbar(cax)
plt.title("Korelacja cech między sobą")
plt.tight_layout()
plt.show()

```



```

In [14]: selector = SelectKBest(score_func=f_regression, k=4)
selector.fit(X, y)

f_scores = selector.scores_
p_values = selector.pvalues_
mi_scores = mutual_info_regression(X, y, random_state=42)

feature_scores = pd.DataFrame({
    "feature": features,
    "correlation": corr_with_score.values,
    "f_regression_score": f_scores,
    "p_value": p_values,
    "mutual_info_score": mi_scores,
})

feature_scores["correlation_rank"] = feature_scores["correlation"].rank(ascending=

```

```

feature_scores["f_regression_rank"] = feature_scores["f_regression_score"].rank(
feature_scores["mutual_info_rank"] = feature_scores["mutual_info_score"].rank(as
feature_scores["mean_rank"] = feature_scores[["correlation_rank", "f_regression_

feature_scores = feature_scores.sort_values(by="mean_rank")
display(feature_scores.round(2))

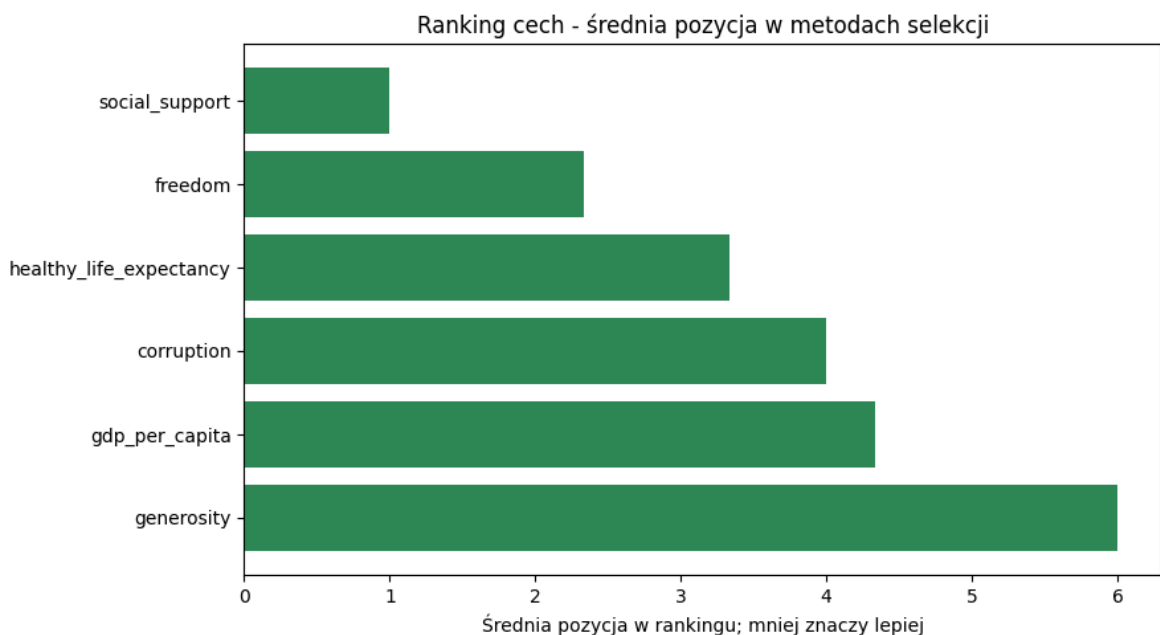
```

	feature	correlation	f_regression_score	p_value	mutual_info_score	correlation_rank
1	social_support	0.98	2623.78	0.00	1.86	1
3	freedom	0.93	859.78	0.00	1.11	2
2	healthy_life_expectancy	0.90	549.32	0.00	1.07	3
5	corruption	0.87	417.12	0.00	1.29	4
0	gdp_per_capita	0.88	436.23	0.00	0.96	5
4	generosity	0.24	7.61	0.01	0.19	6

```

In [15]: plt.figure(figsize=(9, 5))
plt.barh(feature_scores["feature"], feature_scores["mean_rank"], color="seagreen")
plt.gca().invert_yaxis()
plt.title("Ranking cech - średnia pozycja w metodach selekcji")
plt.xlabel("Średnia pozycja w rankingu; mniej znaczy lepiej")
plt.tight_layout()
plt.show()

```



Dodatkowa analiza klasyfikacyjna - f_classif

Ponieważ głównym problemem jest regresja, podstawowym testem pozostaje

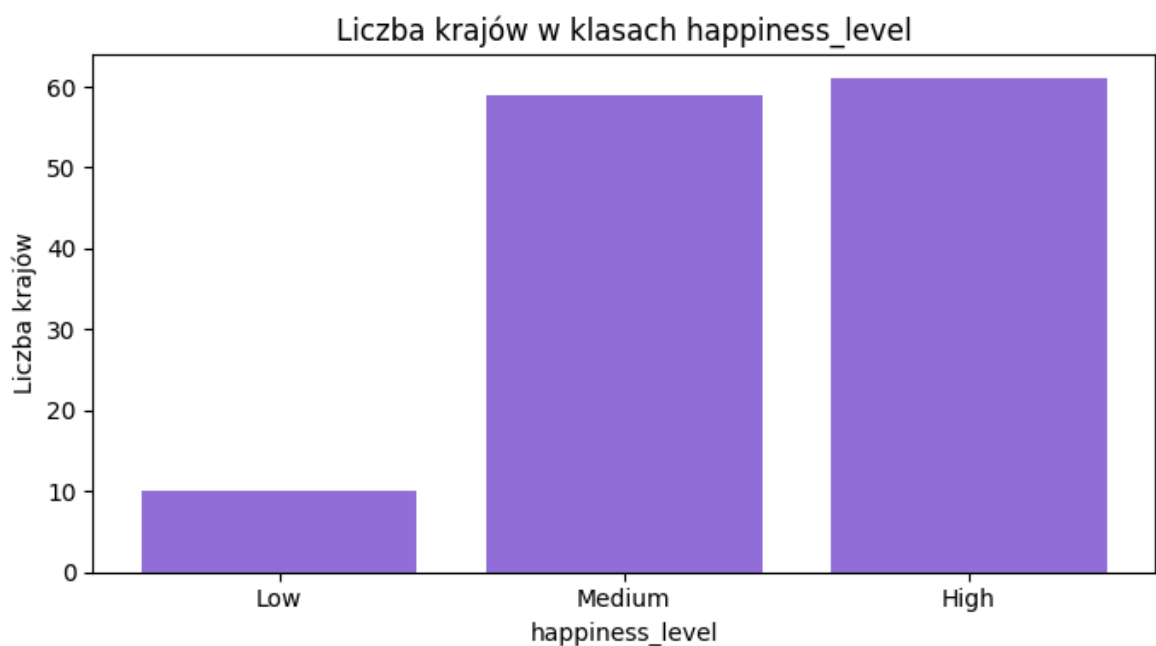
`f_regression`. Dodatkowo można jednak potraktować `score` jako trzy klasy: `Low`, `Medium` i `High`, a następnie sprawdzić, które cechy najlepiej rozróżniają te grupy metodą `f_classif`.

```
In [16]: df["happiness_level"] = pd.cut(
    df["score"],
    bins=3,
    labels=["Low", "Medium", "High"]
)

class_counts = df["happiness_level"].value_counts().sort_index()
display(class_counts.to_frame(name="country_count"))

plt.figure(figsize=(7, 4))
plt.bar(class_counts.index.astype(str), class_counts.values, color="mediumpurple")
plt.title("Liczba krajów w klasach happiness_level")
plt.xlabel("happiness_level")
plt.ylabel("Liczba krajów")
plt.tight_layout()
plt.show()
```

country_count	
happiness_level	
Low	10
Medium	59
High	61



```
In [17]: selector_classif = SelectKBest(score_func=f_classif, k="all")
selector_classif.fit(X, df["happiness_level"])

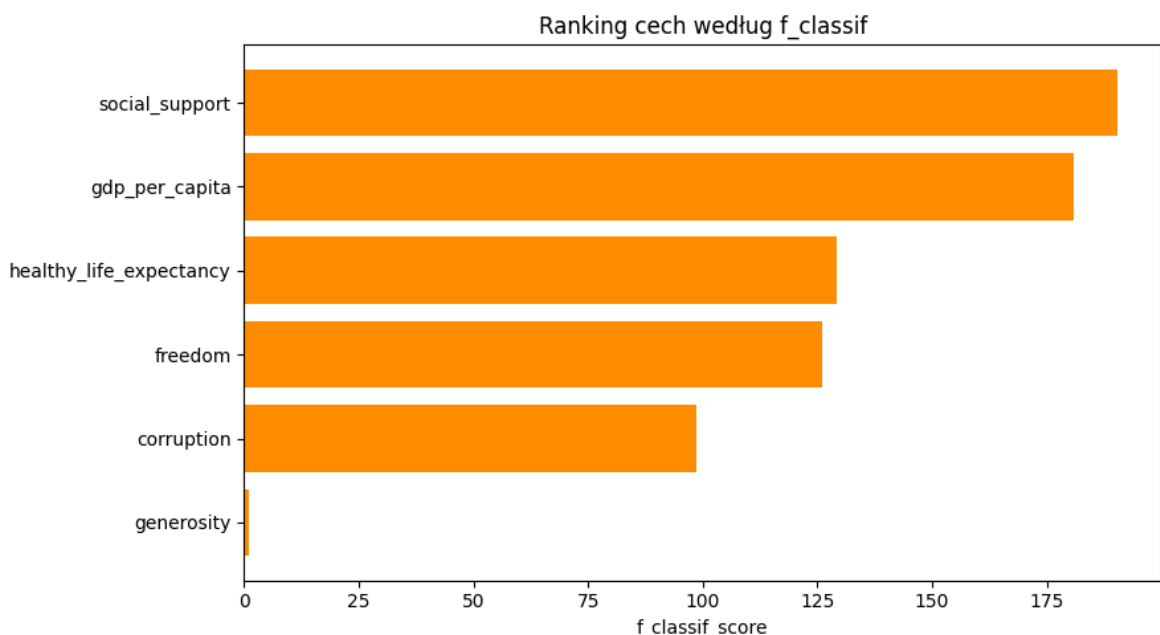
classif_scores = pd.DataFrame({
    "feature": features,
    "f_classif_score": selector_classif.scores_,
    "p_value": selector_classif.pvalues_,
})

classif_scores["f_classif_rank"] = classif_scores["f_classif_score"].rank(ascending=True)
classif_scores = classif_scores.sort_values(by="f_classif_rank")
```

```
display(classif_scores.round(2))
```

	feature	f_classif_score	p_value	f_classif_rank
1	social_support	190.42	0.00	1.0
0	gdp_per_capita	180.76	0.00	2.0
2	healthy_life_expectancy	129.15	0.00	3.0
3	freedom	125.98	0.00	4.0
5	corruption	98.58	0.00	5.0
4	generosity	1.11	0.33	6.0

```
In [18]: plt.figure(figsize=(9, 5))
plt.barh(classif_scores["feature"], classif_scores["f_classif_score"], color="darkorange")
plt.gca().invert_yaxis()
plt.title("Ranking cech według f_classif")
plt.xlabel("f_classif_score")
plt.tight_layout()
plt.show()
```



Wnioski z eksperymentu 1

Eksperyment 1 pokazał, że najważniejszą cechą dla przewidywania wyniku szczęścia jest `social_support`. Cecha ta uzyskała pierwsze miejsce we wszystkich zastosowanych metodach oceny. Wysoką zależność ze `score` wykazują także `freedom`, `healthy_life_expectancy`, `corruption` oraz `gdp_per_capita`. Wszystkie te cechy mają wysoką dodatnią korelację ze `score` i bardzo niskie wartości `p_value`, co wskazuje na istotną statystycznie zależność.

Najsłabszą cechą okazała się `generosity`. Pomimo że jej `p_value` wskazuje na istotność statystyczną, jej korelacja i wyniki rankingowe są wyraźnie niższe niż w

przypadku pozostałych zmiennych. W dalszej części projektu można przygotować dwa warianty danych: pierwszy wykorzystujący wszystkie cechy oraz drugi bez `generosity`.

`f_classif` pokazuje, które cechy najlepiej odróżniają kraje z niskim, średnim i wysokim poziomem szczęścia. Wynik ten należy traktować jako analizę pomocniczą, ponieważ docelowo przewidywana jest wartość liczbową `score`, a nie klasa `happiness_level`.

Przygotowanie danych do modelu

Celem etapu jest przygotowanie zbioru danych do trenowania modeli regresyjnych. Zmienną docelową jest `score`, a cechami są wskaźniki społeczno-ekonomiczne.

Na podstawie eksperymentu 1 przygotowane zostaną dwa warianty danych:

- wariant ze wszystkimi cechami,
- wariant bez cechy `generosity`, która miała najslabszy związek ze `score`.

```
In [19]: features_all = [
    "gdp_per_capita",
    "social_support",
    "healthy_life_expectancy",
    "freedom",
    "generosity",
    "corruption"
]

features_without_generosity = [
    "gdp_per_capita",
    "social_support",
    "healthy_life_expectancy",
    "freedom",
    "corruption"
]

X_all = df[features_all]
X_without_generosity = df[features_without_generosity]
y = df["score"]
```

```
In [20]: print("Braki danych w X_all:")
print(X_all.isnull().sum())

print("\nBraki danych w y:")
print(y.isnull().sum())

print("\nRozmiar X_all:", X_all.shape)
print("Rozmiar X_without_generosity:", X_without_generosity.shape)
print("Rozmiar y:", y.shape)
```

```
Braki danych w X_all:
gdp_per_capita          0
social_support          0
healthy_life_expectancy 0
freedom                 0
generosity              0
corruption              0
dtype: int64
```

```
Braki danych w y:
0
```

```
Rozmiar X_all: (130, 6)
Rozmiar X_without_generosity: (130, 5)
Rozmiar y: (130,)
```

Eksperyment 2 - regresja

Celem eksperymentu jest porównanie kilku modeli regresyjnych przewidujących wartość `score`. Porównane zostaną modele liniowe oraz modele drzewiaste.

Testowane będą dwa warianty danych:

- wszystkie cechy,
- cechy bez `generosity`.

```
In [21]: from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=0.01, max_iter=10000),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42)
}
```

W eksperymencie użyto modeli regresyjnych:

- Linear Regression jako model bazowy,
- Ridge Regression i Lasso Regression jako modele liniowe z regularyzacją,
- Decision Tree Regressor jako prosty model nieliniowy,
- Random Forest Regressor jako model zespołowy oparty na wielu drzewach.

Modele liniowe korzystają ze skalowania cech, dlatego zastosowano `StandardScaler` w potoku `Pipeline`.

Walidacja modeli metodą Leave-One-Out

W kolejnym kroku modele zostaną ocenione za pomocą walidacji krzyżowej Leave-One-Out. Ponieważ zbiór danych jest mały i zawiera 130 krajów, ta metoda pozwala

wykorzystać prawie wszystkie dane do trenowania modelu.

W każdej iteracji model będzie trenowany na 129 krajach, a testowany na 1 kraju. Proces zostanie powtórzony 130 razy, tak aby każdy kraj został raz użyty jako przykład testowy.

```
In [22]: from sklearn.model_selection import LeaveOneOut, cross_val_predict
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

loo = LeaveOneOut()

def evaluate_models(X, y, dataset_name):
    results = []

    for model_name, model in models.items():
        pipeline = Pipeline([
            ("scaler", StandardScaler()),
            ("model", model)
        ])

        y_pred = cross_val_predict(pipeline, X, y, cv=loo)

        mae = mean_absolute_error(y, y_pred)
        rmse = np.sqrt(mean_squared_error(y, y_pred))
        r2 = r2_score(y, y_pred)

        results.append({
            "dataset": dataset_name,
            "model": model_name,
            "MAE": mae,
            "RMSE": rmse,
            "R2": r2
        })

    return pd.DataFrame(results)
```

Ocena modeli za pomocą MAE, RMSE i R^2

Do oceny modeli wykorzystano trzy metryki:

- MAE, czyli średni błąd bezwzględny,
- RMSE, czyli pierwiastek ze średniego błędu kwadratowego,
- R^2 , czyli miarę dopasowania modelu do danych.

Niższe wartości MAE i RMSE oznaczają mniejszy błąd predykcji. Wyższa wartość R^2 oznacza lepsze dopasowanie modelu do danych.

```
In [23]: results_all = evaluate_models(X_all, y, "all_features")
results_without_generosity = evaluate_models(
    X_without_generosity,
    y,
    "without_generosity"
)

results = pd.concat([results_all, results_without_generosity], ignore_index=True)
results = results.sort_values(by="RMSE").reset_index(drop=True)
```

```
display(results)
```

	dataset	model	MAE	RMSE	R2
0	all_features	Linear Regression	0.161602	0.214958	0.970467
1	all_features	Ridge Regression	0.160941	0.219737	0.969140
2	without_generosity	Linear Regression	0.169153	0.222729	0.968293
3	without_generosity	Ridge Regression	0.169926	0.227531	0.966912
4	without_generosity	Lasso Regression	0.178102	0.238953	0.963506
5	all_features	Lasso Regression	0.178252	0.239159	0.963443
6	all_features	Random Forest	0.188283	0.305147	0.940487
7	without_generosity	Random Forest	0.190941	0.306980	0.939770
8	all_features	Decision Tree	0.202854	0.321733	0.933841
9	without_generosity	Decision Tree	0.232285	0.363843	0.915390

Analiza błędów

Po porównaniu modeli do dalszej analizy wybrano Linear Regression trenowany na wszystkich cechach. Model ten uzyskał najlepsze wyniki według metryk MAE, RMSE i R^2 .

W tej części zostaną sprawdzone błędy predykcji najlepszego modelu. Analiza pozwoli zobaczyć, dla których krajów model pomylił się najbardziej oraz czy wielkość błędu różni się między pomocniczymi klasami `happiness_level`.

```
In [24]: best_model = Pipeline([
          ("scaler", StandardScaler()),
          ("model", LinearRegression())
        ])

y_pred_best = cross_val_predict(best_model, X_all, y, cv=loo)

errors = df[["country", "region", "score", "happiness_level"]].copy()
errors["predicted_score"] = y_pred_best
errors["error"] = errors["score"] - errors["predicted_score"]
errors["absolute_error"] = errors["error"].abs()

errors_sorted = errors.sort_values(by="absolute_error", ascending=False)

display(errors_sorted.head(10))
```

	country	region	score	happiness_level	predicted_score	error	abso
108	Rwanda	Sub-Saharan Africa	3.911	Medium	4.668263	-0.757263	
15	Kosovo	Central and Eastern Europe	6.910	High	6.314099	0.595901	
36	Hungary	Central and Eastern Europe	6.281	High	5.696822	0.584178	
3	Costa Rica	Latin America and Caribbean	7.439	High	6.954785	0.484215	
129	Afghanistan	South Asia	1.446	Low	0.974484	0.471516	
122	Lesotho	Sub-Saharan Africa	3.501	Low	3.958775	-0.457775	
85	Mauritius	Sub-Saharan Africa	5.171	Medium	5.612614	-0.441614	
65	Tajikistan	Commonwealth of Independent States	5.571	Medium	5.992272	-0.421272	
6	Netherlands	Western Europe	7.319	High	6.918286	0.400714	
126	South Sudan	Sub-Saharan Africa	3.421	Low	3.027928	0.393072	

Największy błąd model uzyskał dla Rwandy. Rzeczywisty wynik `score` wynosił 3.911, natomiast model przewidział 4.668263. Oznacza to, że model zawyżył wynik o około 0.76 punktu.

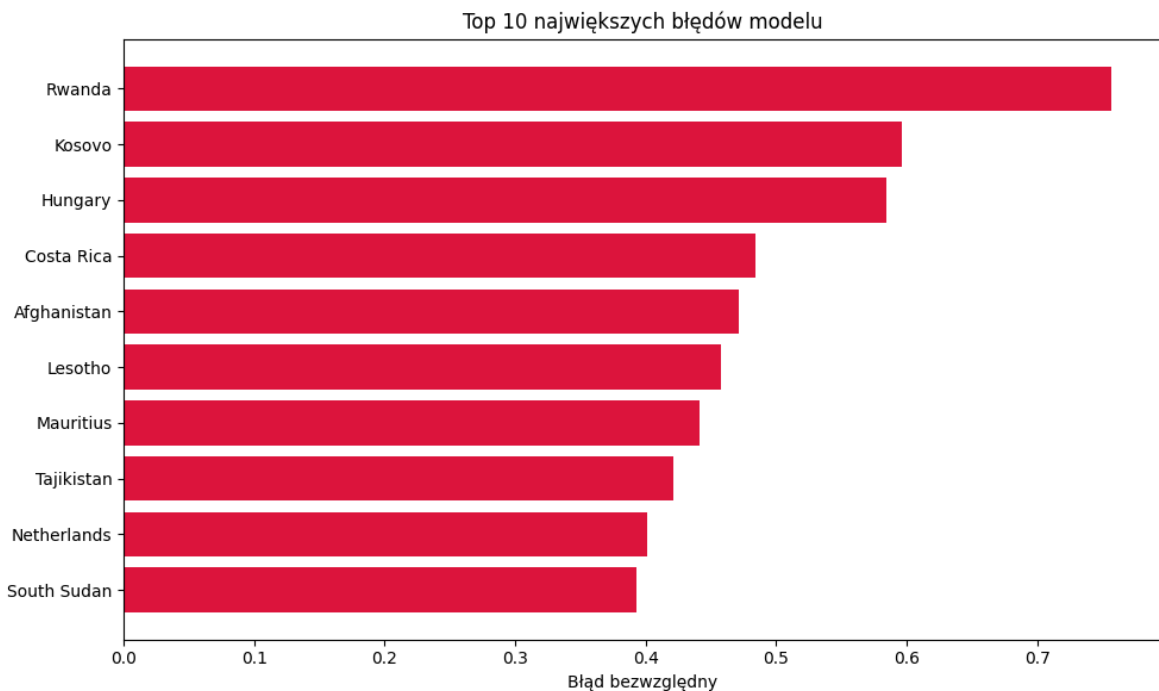
Duże błędy pojawiły się również dla Kosowa, Węgier, Kostaryki oraz Afganistanu. W przypadku Kosowa, Węgier i Kostaryki model przewidział wynik niższy niż rzeczywisty, natomiast dla Rwandy, Lesotho, Mauritiusa i Tadżykistanu model zawyżył wynik.

Większość największych błędów dotyczy krajów z klas `High` oraz `Medium`, ale wśród nich pojawiają się także kraje z klasy `Low`, takie jak Afganistan, Lesotho i Sudan Południowy.

Oznacza to, że błędy nie występują wyłącznie w jednej grupie poziomu szczęścia.

```
In [25]: top_errors = errors_sorted.head(10)

plt.figure(figsize=(10, 6))
plt.barh(top_errors["country"], top_errors["absolute_error"], color="crimson")
plt.gca().invert_yaxis()
plt.title("Top 10 największych błędów modelu")
plt.xlabel("Błąd bezwzględny")
plt.tight_layout()
plt.show()
```



Porównanie z najgorszym modelem

Postanowiono wykonać porównanie błędów najlepszego modelu z najgorszym modelem.

```
In [26]: worst_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", DecisionTreeRegressor(random_state=42))
])

y_pred_worst = cross_val_predict(worst_model, X_without_generosity, y, cv=loo)

worst_errors = df[["country", "region", "score", "happiness_level"]].copy()
worst_errors["predicted_score"] = y_pred_worst
worst_errors["error"] = worst_errors["score"] - worst_errors["predicted_score"]
worst_errors["absolute_error"] = worst_errors["error"].abs()

worst_errors_sorted = worst_errors.sort_values(by="absolute_error", ascending=False)
```

```
In [27]: model_error_comparison = errors[["country", "absolute_error"]].rename(
    columns={"absolute_error": "linear_absolute_error"}
)

decision_tree_errors = worst_errors[["country", "absolute_error"]].rename(
    columns={"absolute_error": "decision_tree_absolute_error"}
)

model_error_comparison = model_error_comparison.merge(
    decision_tree_errors,
    on="country"
)

model_error_comparison["difference"] = (
    model_error_comparison["decision_tree_absolute_error"]
    - model_error_comparison["linear_absolute_error"]
)

model_error_comparison = model_error_comparison.sort_values(
```

```

        by="difference",
        ascending=False
    ).reset_index(drop=True)

mean_row = pd.DataFrame({
    "country": ["ŚREDNI BŁĄD"],
    "linear_absolute_error": [model_error_comparison["linear_absolute_error"].mean()],
    "decision_tree_absolute_error": [model_error_comparison["decision_tree_absolute_error"].mean()],
    "difference": [model_error_comparison["difference"].mean()]
})

model_error_comparison_with_mean = pd.concat(
    [model_error_comparison, mean_row],
    ignore_index=True
)

display(model_error_comparison_with_mean)

```

	country	linear_absolute_error	decision_tree_absolute_error	difference
0	Lebanon	0.213582	1.250000	1.036418
1	Afghanistan	0.471516	1.315000	0.843484
2	Tanzania	0.266466	1.080000	0.813534
3	Togo	0.140528	0.900000	0.759472
4	Malaysia	0.162955	0.920000	0.757045
5	Mexico	0.304529	1.061000	0.756471
6	Congo DR	0.010183	0.680000	0.669817
7	Kosovo	0.595901	1.139000	0.543099
8	Burkina Faso	0.260313	0.800000	0.539687
9	Slovakia	0.142324	0.570000	0.427676
10	United Arab Emirates	0.204116	0.588000	0.383884
11	Singapore	0.386910	0.740000	0.353090
12	Palestinian Territories	0.033180	0.360000	0.326820
13	Sierra Leone	0.020645	0.330000	0.309355
14	Poland	0.032841	0.340000	0.307159
15	Estonia	0.016790	0.320000	0.303210
16	Rwanda	0.757263	1.060000	0.302737
17	France	0.010686	0.285000	0.274314
18	Costa Rica	0.484215	0.758000	0.273785
19	Luxembourg	0.145195	0.417000	0.271805
20	Saudi Arabia	0.367754	0.630000	0.262246
21	Myanmar	0.105795	0.360000	0.254205
22	Chile	0.031274	0.280000	0.248726
23	Sudan	0.025737	0.270000	0.244263
24	Sri Lanka	0.041553	0.280000	0.238447
25	Ethiopia	0.003686	0.240000	0.236314
26	Somalia	0.018504	0.250000	0.231496
27	Israel	0.067214	0.286000	0.218786
28	Norway	0.161423	0.372000	0.210577
29	United Kingdom	0.001811	0.200000	0.198189
30	Mongolia	0.014684	0.200000	0.185316
31	Switzerland	0.027369	0.191000	0.163631

	country	linear_absolute_error	decision_tree_absolute_error	difference
32	Australia	0.072224	0.233000	0.160776
33	Japan	0.298292	0.456000	0.157708
34	Pakistan	0.028867	0.180000	0.151133
35	Burundi	0.099456	0.250000	0.150544
36	China	0.229816	0.378000	0.148184
37	Guinea-Bissau	0.104222	0.250000	0.145778
38	Portugal	0.094325	0.240000	0.145675
39	Thailand	0.056787	0.190000	0.133213
40	Jordan	0.007307	0.140000	0.132693
41	Ireland	0.286566	0.417000	0.130434
42	Cameroon	0.062436	0.180000	0.117564
43	South Korea	0.182905	0.300000	0.117095
44	Uruguay	0.187282	0.300000	0.112718
45	Iraq	0.050407	0.160000	0.109593
46	Yemen	0.161028	0.270000	0.108972
47	Latvia	0.212048	0.320000	0.107952
48	Netherlands	0.400714	0.508000	0.107286
49	Tunisia	0.021511	0.120000	0.098489
50	Algeria	0.060976	0.140000	0.079024
51	Kazakhstan	0.013664	0.090000	0.076336
52	Mauritania	0.106767	0.180000	0.073233
53	Bangladesh	0.035649	0.080000	0.044351
54	Denmark	0.032313	0.076000	0.043687
55	Bolivia	0.058881	0.100000	0.041119
56	Panama	0.019563	0.060000	0.040437
57	DR Congo	0.231616	0.270000	0.038384
58	Peru	0.026512	0.060000	0.033488
59	Czechia	0.160582	0.194000	0.033418
60	Taiwan	0.175180	0.207000	0.031820
61	Nigeria	0.010667	0.040000	0.029333
62	Haiti	0.030963	0.060000	0.029037
63	Honduras	0.074507	0.100000	0.025493
64	Liberia	0.017873	0.040000	0.022127

	country	linear_absolute_error	decision_tree_absolute_error	difference
65	Russia	0.029744	0.050000	0.020256
66	Finland	0.057817	0.076000	0.018183
67	Libya	0.013180	0.020000	0.006820
68	Slovenia	0.013651	0.017000	0.003349
69	Niger	0.037677	0.040000	0.002323
70	Ecuador	0.059217	0.060000	0.000783
71	Ivory Coast	0.039695	0.040000	0.000305
72	Benin	0.085166	0.080000	-0.005166
73	Austria	0.052426	0.036000	-0.016426
74	Congo	0.147884	0.130000	-0.017884
75	Uganda	0.083884	0.060000	-0.023884
76	South Africa	0.104834	0.080000	-0.024834
77	Egypt	0.088150	0.060000	-0.028150
78	Guinea	0.149607	0.120000	-0.029607
79	Kenya	0.055648	0.020000	-0.035648
80	India	0.060222	0.020000	-0.040222
81	Canada	0.292426	0.250000	-0.042426
82	Gabon	0.083341	0.040000	-0.043341
83	Belgium	0.090701	0.047000	-0.043701
84	Ghana	0.065272	0.020000	-0.045272
85	Laos	0.129513	0.080000	-0.049513
86	United States	0.185324	0.135000	-0.050324
87	Belarus	0.072597	0.020000	-0.052597
88	Turkey	0.093526	0.040000	-0.053526
89	Angola	0.134818	0.080000	-0.054818
90	Nicaragua	0.155358	0.100000	-0.055358
91	Germany	0.094108	0.036000	-0.058108
92	Colombia	0.101460	0.040000	-0.061460
93	Mauritius	0.441614	0.380000	-0.061614
94	Indonesia	0.082448	0.020000	-0.062448
95	Zimbabwe	0.302879	0.240000	-0.062879
96	Spain	0.083231	0.020000	-0.063231
97	Cambodia	0.103290	0.040000	-0.063290

	country	linear_absolute_error	decision_tree_absolute_error	difference
98	Morocco	0.083928	0.020000	-0.063928
99	Senegal	0.085137	0.020000	-0.065137
100	Italy	0.209743	0.140000	-0.069743
101	Madagascar	0.113548	0.040000	-0.073548
102	Sweden	0.157840	0.082000	-0.075840
103	Paraguay	0.187406	0.100000	-0.087406
104	Lesotho	0.457775	0.370000	-0.087775
105	Central African Republic	0.112746	0.020000	-0.092746
106	Hungary	0.584178	0.490000	-0.094178
107	Eswatini	0.208280	0.100000	-0.108280
108	Botswana	0.254845	0.140000	-0.114845
109	Philippines	0.181010	0.060000	-0.121010
110	Chad	0.163881	0.040000	-0.123881
111	Nepal	0.150149	0.020000	-0.130149
112	Mozambique	0.226343	0.080000	-0.146343
113	Greece	0.351684	0.200000	-0.151684
114	Iceland	0.169708	0.013000	-0.156708
115	Namibia	0.305506	0.140000	-0.165506
116	Malawi	0.258348	0.080000	-0.178348
117	Comoros	0.239846	0.060000	-0.179846
118	Vietnam	0.268711	0.080000	-0.188711
119	Brazil	0.254695	0.040000	-0.214695
120	Lithuania	0.236178	0.017000	-0.219178
121	Romania	0.302849	0.070000	-0.232849
122	Argentina	0.279426	0.040000	-0.239426
123	Mali	0.287153	0.040000	-0.247153
124	Zambia	0.304442	0.040000	-0.264442
125	Kyrgyzstan	0.308559	0.020000	-0.288559
126	New Zealand	0.358613	0.017000	-0.341613
127	Serbia	0.361818	0.020000	-0.341818
128	South Sudan	0.393072	0.020000	-0.373072
129	Tajikistan	0.421272	0.020000	-0.401272

	country	linear_absolute_error	decision_tree_absolute_error	difference
130	ŚREDNI BŁĄD	0.161602	0.232285	0.070683

Porównanie błędów najlepszego i najgorszego modelu pokazuje, że Linear Regression popełniał średnio mniejsze błędy niż Decision Tree. Średni błąd bezwzględny Linear Regression wyniósł 0.161602, natomiast dla Decision Tree wyniósł 0.232285.

Oznacza to, że Decision Tree mylił się średnio o około 0.07 punktu `score` bardziej niż Linear Regression. Dla części krajów Decision Tree uzyskał mniejszy błąd, jednak ogólnie Linear Regression był bardziej stabilnym modelem w całym zbiorze danych.

Wyniki te potwierdzają wcześniejszą ocenę modeli na podstawie MAE, RMSE i R^2 .

Test statystyczny

W celu sprawdzenia, czy różnica między najlepszym i najgorszym modelem jest istotna statystycznie, zastosowano test Wilcoxa dla prób zależnych. Porównano błędy bezwzględne obu modeli dla tych samych krajów.

Najpierw wykonano test Shapiro-Wilka, aby sprawdzić normalność rozkładu różnic błędów między modelami.

```
In [28]: from scipy.stats import shapiro, wilcoxon

alpha = 0.05

differences = (
    model_error_comparison["decision_tree_absolute_error"]
    - model_error_comparison["linear_absolute_error"]
)

shapiro_stat, shapiro_p = shapiro(differences)

wilcoxon_stat, wilcoxon_p = wilcoxon(
    model_error_comparison["decision_tree_absolute_error"],
    model_error_comparison["linear_absolute_error"],
    alternative="greater"
)

statistical_test_results = pd.DataFrame({
    "test": ["Shapiro-Wilk", "Wilcoxon signed-rank"],
    "statistic": [shapiro_stat, wilcoxon_stat],
    "p_value": [shapiro_p, wilcoxon_p],
    "alpha": [alpha, alpha]
})

display(statistical_test_results)

if wilcoxon_p < alpha:
    print("Różnica błędów jest istotna statystycznie.")
    print("Decision Tree ma istotnie większe błędy bezwzględne niż Linear Regres
else:
    print("Nie ma podstaw do stwierdzenia istotnej statystycznie różnicy błędów.")
```

	test	statistic	p_value	alpha
0	Shapiro-Wilk	0.899381	7.171037e-08	0.05
1	Wilcoxon signed-rank	5221.000000	1.258204e-02	0.05

Różnica błędów jest istotna statystycznie.

Decision Tree ma istotnie większe błędy bezwzględne niż Linear Regression.

Test Shapiro-Wilka wykazał, że rozkład różnic błędów odbiega istotnie od rozkładu normalnego, $p < 0.001$. Z tego względu do porównania modeli zastosowano nieparametryczny test Wilcoxona dla prób zależnych. Wynik testu Wilcoxona był istotny statystycznie, $W = 5221$, $p = 0.0126 < 0.05$. Oznacza to, że model Decision Tree uzyskuje istotnie większe błędy bezwzględne niż Linear Regression.

Eksperyment 3 - Klasteryzacja

Eksperyment przeprowadzono, aby znaleźć naturalne grupy krajów o podobnych cechach społeczno-ekonomicznych. Sprawdzone, czy dane same "mówią", że istnieją segmenty krajów o zbliżonym profilu.

Klasteryzacja pokaże, które kraje są podobne do siebie pod względem cech.

Użyto wszystkich cech (`features_all`), gdyż analizując poprzednie eksperymenty widać, że nie ma potrzeby wykluczać cechy `generosity`.

Najpierw zeskalowano dane, a następnie użyto metody łokcia, aby sprawdzić optymalną liczbę klastrów.

SSE - Sum of Squared Errors

```
In [34]: from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

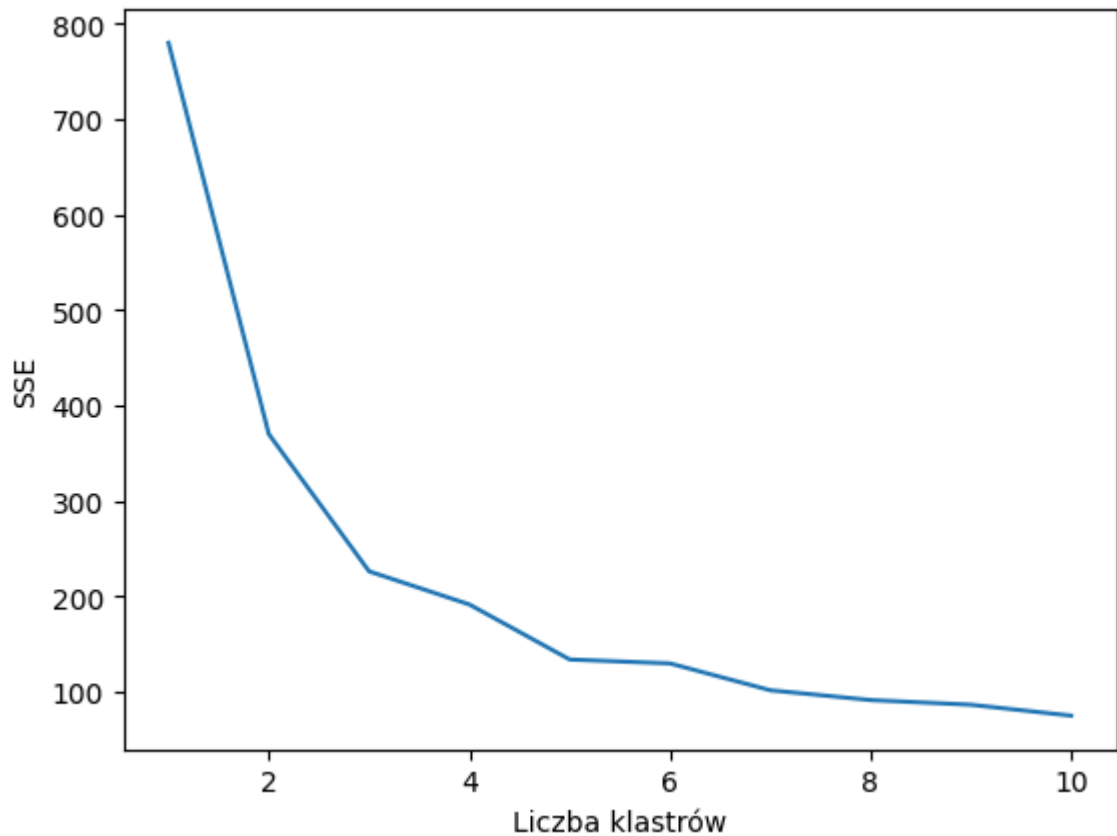
X_c = df[features_all]

# Skalowanie danych
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_c)

SSE = []

for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(X_scaled)
    SSE.append(model.inertia_)

plt.plot(range(1, 11), SSE)
plt.xlabel("Liczba klastrów")
plt.ylabel("SSE")
plt.show()
```



Zastosowano liczbę klastrów równą 3.

Klasteryzację przeprowadzono przy użyciu `KMeans` i zwizualizowano redukując wymiary za pomocą `PCA`.

```
In [35]: from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Klasteryzacja używając kmeans
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

df['cluster'] = clusters

# Redukcja wymiarowości
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
order = df.groupby("cluster")["score"].mean().sort_values().index

cluster_map = {
    order[0]: "Low",
    order[1]: "Medium",
    order[2]: "High"
}

df["cluster_label"] = df["cluster"].map(cluster_map)

plt.figure(figsize=(8,6))

colors = {0: "tab:blue", 1: "tab:orange", 2: "tab:green"}
```

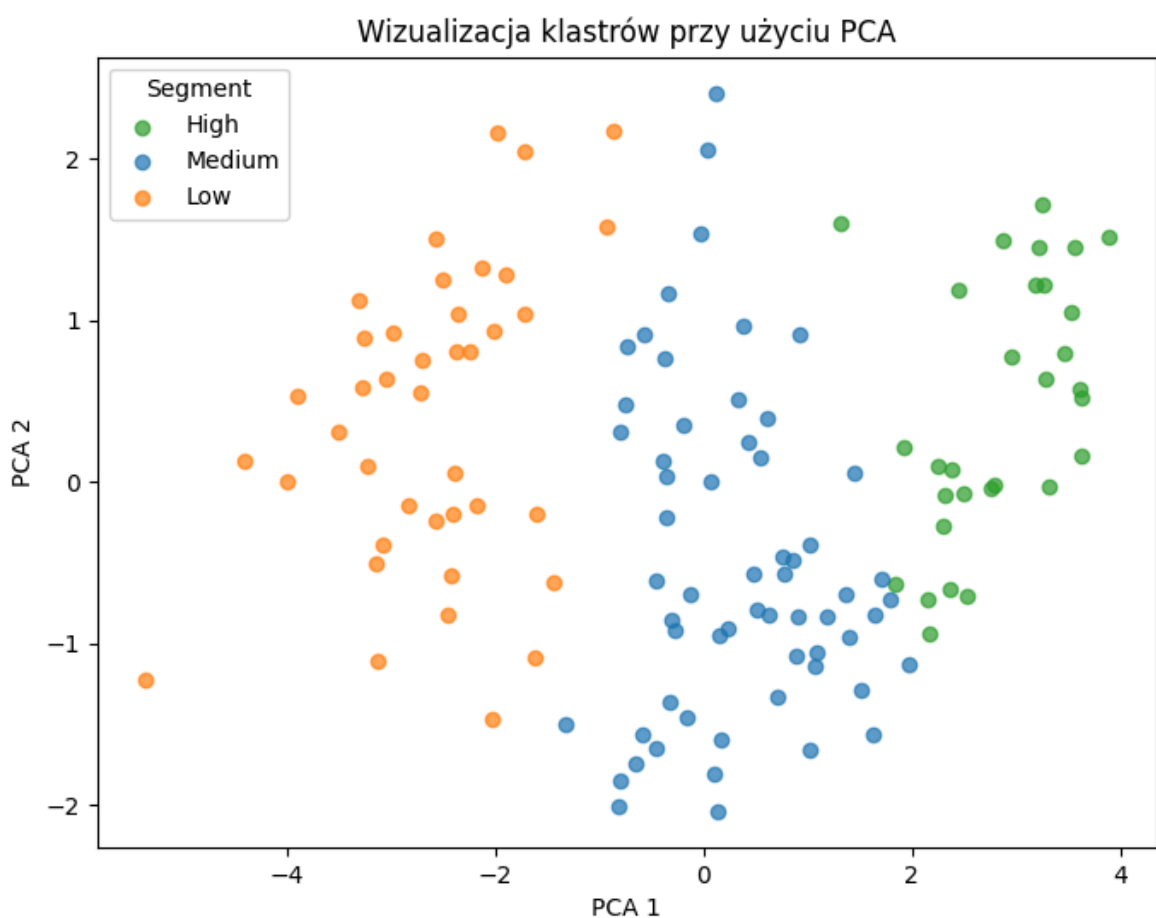
```

for cluster_id in df["cluster"].unique():
    mask = df["cluster"] == cluster_id

    plt.scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        c=colors[cluster_id],
        label=cluster_map[cluster_id],
        alpha=0.7
    )

plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.title("Wizualizacja klastrów przy użyciu PCA")
plt.legend(title="Segment")
plt.show()

```



Porównanie klastrów z `happiness_level` i metryka separacji

Poniższy kod pokazuje, jak grupy z klasteryzacji K-Means pokrywają się z pomocniczym podziałem `happiness_level`, a także liczy współczynnik silhouette dla `k=3`, aby ocenić jakość podziału.

```

In [36]: from sklearn.metrics import silhouette_score

if "happiness_level" not in df.columns:
    df["happiness_level"] = pd.cut(df["score"], bins=3, labels=["Low", "Medium",

```

```

ordered_labels = ["High", "Medium", "Low"]
df["cluster_label"] = pd.Categorical(df["cluster_label"], categories=ordered_labels)

print("Tabela kontyngencji klastra vs happiness_level:")
ct = pd.crosstab(df["cluster_label"], df["happiness_level"]).reindex(index=ordered_labels, columns=ordered_labels)
display(ct)

print("Rozmiary klastrów:")
display(df["cluster_label"].value_counts().reindex(ordered_labels))

sil = silhouette_score(X_scaled, kmeans.labels_)
print(f"Silhouette score dla k=3: {sil:.3f}")

cluster_means = df.groupby("cluster_label")[["score"] + features_all].mean().reindex(ordered_labels)
print("Średnie wartości cech w klastrach:")
display(cluster_means.round(3))

```

Tabela kontyngencji klastra vs happiness_level:

happiness_level **High** **Medium** **Low**

cluster_label				
	High	Medium	Low	
	30	0	0	
	31	29	0	
Low	0	30	10	

Rozmiary klastrów:

cluster_label

High 30

Medium 60

Low 40

Name: count, dtype: int64

Silhouette score dla k=3: 0.419

Średnie wartości cech w klastrach:

	score	gdp_per_capita	social_support	healthy_life_expectancy	freedom	ger
cluster_label						
High	6.949	1.828	1.471	0.924	0.628	
Medium	5.742	1.222	1.255	0.804	0.498	
Low	3.900	0.486	0.839	0.479	0.353	



```

In [38]: display(df[["country", "score", "cluster_label"]].sort_values("score", ascending=False).reset_index(drop=True))

```

	country	score	cluster_label
0	Finland	7.764	High
1	Iceland	7.701	High
2	Denmark	7.688	High
3	Costa Rica	7.439	High
4	Sweden	7.401	High
5	Norway	7.392	High
6	Netherlands	7.319	High
7	Israel	7.234	High
8	Luxembourg	7.228	High
9	Switzerland	7.201	High
10	Australia	7.181	High
11	Mexico	6.972	Medium
12	New Zealand	6.948	High
13	Germany	6.941	High
14	Canada	6.931	High
15	Kosovo	6.910	Medium
16	Austria	6.905	High
17	Belgium	6.894	High
18	Czechia	6.888	High
19	Slovenia	6.871	High
20	United Arab Emirates	6.851	High
21	Saudi Arabia	6.831	High
22	United States	6.816	High
23	Ireland	6.811	High
24	United Kingdom	6.694	High
25	Taiwan	6.681	High
26	France	6.586	High
27	Poland	6.541	High
28	Estonia	6.521	High
29	Lithuania	6.498	Medium
30	Slovakia	6.481	Medium
31	Latvia	6.421	Medium
32	Spain	6.401	Medium

	country	score	cluster_label
33	Italy	6.381	Medium
34	Romania	6.351	Medium
35	Serbia	6.301	Medium
36	Hungary	6.281	Medium
37	Portugal	6.241	Medium
38	Greece	6.221	Medium
39	Singapore	6.201	High
40	Japan	6.130	High
41	South Korea	6.101	Medium
42	Brazil	6.081	Medium
43	Argentina	6.041	Medium
44	Chile	6.021	Medium
45	Uruguay	6.001	Medium
46	China	5.973	Medium
47	Russia	5.921	Medium
48	Malaysia	5.911	Medium
49	Thailand	5.891	High
50	Belarus	5.871	Medium
51	Colombia	5.851	Medium
52	Kazakhstan	5.831	Medium
53	Ecuador	5.811	Medium
54	Panama	5.791	Medium
55	Paraguay	5.771	Medium
56	Peru	5.751	Medium
57	Bolivia	5.731	Medium
58	Nicaragua	5.711	Medium
59	Vietnam	5.691	Medium
60	Philippines	5.671	Medium
61	Indonesia	5.651	Medium
62	Honduras	5.631	Medium
63	Mongolia	5.611	Medium
64	Kyrgyzstan	5.591	Medium
65	Tajikistan	5.571	Medium

	country	score	cluster_label
66	Morocco	5.551	Medium
67	India	5.531	Medium
68	Nepal	5.511	Medium
69	Pakistan	5.491	Medium
70	Bangladesh	5.471	Medium
71	Sri Lanka	5.451	Medium
72	Myanmar	5.431	Medium
73	Cambodia	5.411	Medium
74	Laos	5.391	Medium
75	Kenya	5.371	Medium
76	Ghana	5.351	Medium
77	Nigeria	5.331	Medium
78	Ivory Coast	5.311	Medium
79	Senegal	5.291	Medium
80	Uganda	5.271	Low
81	Algeria	5.251	Medium
82	Jordan	5.231	Medium
83	Turkey	5.211	Medium
84	Egypt	5.191	Medium
85	Mauritius	5.171	Medium
86	South Africa	5.151	Medium
87	Tunisia	5.131	Medium
88	Libya	5.111	Medium
89	Iraq	5.091	Medium
90	Palestinian Territories	5.071	Medium
91	Mozambique	4.971	Low
92	Mali	4.951	Low
93	Burkina Faso	4.931	Low
94	Zambia	4.911	Low
95	Malawi	4.891	Low
96	Chad	4.151	Low
97	Ethiopia	4.131	Low
98	Niger	4.111	Low

	country	score	cluster_label
99	Madagascar	4.091	Low
100	Guinea	4.071	Low
101	Liberia	4.051	Low
102	Togo	4.031	Low
103	Comoros	4.011	Low
104	Benin	3.991	Low
105	Gabon	3.971	Low
106	Cameroon	3.951	Low
107	Botswana	3.931	Low
108	Rwanda	3.911	Low
109	Zimbabwe	3.891	Low
110	Tanzania	3.871	Low
111	Eswatini	3.851	Low
112	Congo	3.831	Low
113	Sudan	3.811	Low
114	Namibia	3.791	Low
115	Mauritania	3.771	Low
116	Angola	3.751	Low
117	Somalia	3.731	Low
118	Guinea-Bissau	3.711	Low
119	Lebanon	3.701	Low
120	Yemen	3.541	Low
121	Haiti	3.521	Low
122	Lesotho	3.501	Low
123	Burundi	3.481	Low
124	DR Congo	3.461	Low
125	Central African Republic	3.441	Low
126	South Sudan	3.421	Low
127	Sierra Leone	3.401	Low
128	Congo DR	2.761	Low
129	Afghanistan	1.446	Low

Najwięcej krajów trafiło do klastra "Medium".

Silhouette score wyniósł 0,419, co oznacza, że klastry są umiarkowanie odseparowane i większość punktów znajduje się w poprawnym klastrze.

Z tabeli powyżej widać, że wynik nie jest idealny, ale w większości dane zostały poprawnie skategoryzowane.